Product Requirement Document (PRD)

AI-POWERED CODE REVIEW & BUG SUGGESTION PLATFORM

Document ID:	Author:	Target Release:	Status:
PRD-2026-CR01	Lead Architect / Engineering	v1.0 MVP	Approved for Dev

1. Executive Summary & Vision

Modern software development teams waste significant engineering hours conducting routine manual code reviews, frequently missing subtle edge-case bugs and security vulnerabilities. This project aims to build an enterprise-grade web application that automates initial code audits using AI, delivering instant side-by-side diffs, bug severity tags, and refactoring suggestions.

2. Target Personas & User Roles

User Persona	Primary Needs & Pain Points	Platform Expectation
Individual Developer	Wants instant feedback on code snippets before submitting Pull Requests.	Fast editor UI, clean side-by-side diffs, easy copy-paste refactored code.
Tech Lead / Senior Engineer	Spends 10+ hours/week reviewing PRs; concerned about security leaks.	Automated security scans, severity categorization, team-wide review sharing.
Engineering Manager	Needs visibility into code quality metrics across team repositories.	Team workspaces, review history dashboards, and compliance tracking.

3. High-Level User Journey Flow



> **Product Success Metrics (KPIs)**
> **Target:** Achieve average code review completion under 5 seconds; 85%+ user acceptance rate on suggested refactorings; zero leakage of user secrets to third-party providers.

4. Functional Requirements (FR) Specification

The following core capabilities must be delivered for the v1.0 release, categorized by functional block and execution priority:

FR-1: User Authentication & Organization Management

MUST HAVE

Description: Secure login system allowing developers to register via Email/Password or GitHub OAuth. Users can create or join team workspaces to share reviews.

- JWT-based session authentication with HTTP-only Refresh Tokens stored securely.
- PostgreSQL backend schema enforcing Role-Based Access Control (RBAC: Admin, Member, Viewer).
- Organization workspace boundary ensuring snippets are kept private to team members.

FR-2: Code Ingestion & Interactive Monaco Editor

MUST HAVE

Description: Full-featured web code editor supporting manual pasting, direct drag-and-drop file uploads, and automatic syntax detection.

- Embedded React `@monaco-editor/react` editor component configured with VS Code dark/light themes.
- Automatic language detection based on extension (.ts, .js, .py, .java, .go, .cpp, .py).
- Client-side pre-validation checking file size limits (max 500 KB or 1,500 lines per run).

FR-3: AI Review Engine & Structured Output Generator

MUST HAVE

Description: NestJS backend processing engine that feeds sanitized code into LLMs and receives strictly typed JSON diagnostics.

- Pre-review secret scanner filtering out API keys, passwords, and private tokens prior to API invocation.
- Strict JSON schema enforcement containing overall summary, severity score (1-10), line-by-line issue arrays, and complete refactored source.
- Persistence of raw inputs, issues, and refactored code in MongoDB document collections.

FR-4: Interactive Side-by-Side Diff & Issue Badges

MUST HAVE

Description: UI display allowing developers to visually compare original code against AI-refactored code with inline issue highlights.

- Monaco `DiffEditor` view highlighting exact line insertions, deletions, and modifications side-by-side.
- Severity tags (Critical = Red, High = Orange, Medium = Yellow, Low = Green) anchored to specific line numbers.
- "One-Click Copy" and "Apply Refactor" action buttons to speed up developer workflow.

FR-5: Real-Time SSE Streaming & Async Queueing SHOULD HAVE

Description: Streaming UI updates and background queue execution to keep the interface fast and handle API spikes gracefully.

- NestJS Server-Sent Events (`@Sse`) controller streaming review tokens dynamically to the React client.
- BullMQ + Redis async task queue offloading long-running code evaluations from main HTTP threads.
- Redis caching layer hashing code inputs with SHA-256 to return instant results for duplicate queries.

FR-6: GitHub Webhook & PR Bot Automation COULD HAVE (V1.1)

Description: Automated GitHub integration that triggers code reviews whenever Pull Requests are created or updated.

- `POST /webhooks/github` endpoint listening to `pull_request.opened` triggers.
- Automatic comment posting with line-anchored suggestions via Octokit REST API.

5. Non-Functional Requirements (NFR)

Category	Target Requirement / Benchmark	Enforcement Mechanism
Latency	Sub-second initial response; review stream starts within 1.5 seconds.	SSE streaming + BullMQ Redis queue.
Security	Zero leaks of user credentials or proprietary code to public models.	Local secret regex pre-scanner + Zero Data Retention LLM tier.
Scalability	Handle 100 concurrent code evaluation jobs without thread starvation.	BullMQ worker pool separation from NestJS REST API.
Availability	99.9% API Uptime during business operating hours.	Stateless NestJS containers on AWS ECS / Docker.

6. Dual-Database Data Schema Strategy

PostgreSQL Schema (Structured Data)

- **users**: id, email, password_hash, github_id
- **teams**: id, name, owner_id, plan_tier
- **team_members**: id, team_id, user_id, role
- **rate_limits**: user_id, token_quota_used

MongoDB Schema (Unstructured Data)

- **reviews**: _id, userId, teamId, language
- **originalCode**: String (raw payload)
- **refactoredCode**: String (LLM payload)
- **issues**: Array of {line, severity, fix}

7. Technology Architecture Stack

Layer	Selected Technology	Selection Justification
Frontend	React 18, TypeScript, Tailwind CSS	Component reusability, strict typing, fast styling iteration.
Code Editor	@monaco-editor/react	Industry standard VS Code engine with native diff capabilities.
Backend Framework	NestJS (Node.js)	Enterprise Modular architecture, native SSE and Guard support.
Relational Database	PostgreSQL + Prisma ORM	ACID compliance, robust relation management, type safety.
Document Store	MongoDB + Mongoose	High write throughput for variable-length code JSON logs.

8. Phased Release Roadmap & Ownership

- **Milestone 1: Identity & Database Plumbing (Weeks 1–2)**
  Owner: Backend Team. Complete PostgreSQL auth endpoints, JWT passport guards, and MongoDB schema bindings.
- **Milestone 2: Monaco Frontend & LLM Review Service (Weeks 3–4)**
  Owner: Full-Stack Team. Deliver Monaco editor interface, JSON mode LLM integration, and side-by-side diff renderer.
- **Milestone 3: Queues, Real-Time Streaming & Sharing (Weeks 5–6)**
  Owner: Infrastructure / Frontend Team. Implement BullMQ queueing, SSE token streaming, and team review share links.

9. Risk Management & Mitigation Strategy

- **Risk: LLM Rate Limits & High API Costs**
  Mitigation: Enforce user quota limits in NestJS Throttler; implement SHA-256 Redis caching to prevent duplicate API billing.
- **Risk: LLM Hallucinated Code Suggestions**
  Mitigation: Run linter pre-checks and display disclaimers requiring developer verification before applying code changes.

10. Document Sign-Off & Approvals

Stakeholder Role	Name / Title	Status	Date
Product Lead	Alex Morgan	Approved	Oct 20, 2026
Engineering Lead	David Chen	Approved	Oct 20, 2026
Security Lead	Sarah Jenkins	Approved with Guardrails	Oct 21, 2026